

Bachelor of Information System

IS2204 - Information System Security

Lecture – 4 : Asymmetric Key Cryptography

Kasun De Zoysa
kasun@ucsc.cmb.ac.lk



UNIVERSITY OF COLOMBO SCHOOL OF COMPUTING



Symmetric Key / Private Key Cryptosystem

- Uses a single Private Key shared between users
- Strengths
 - Speed/ Efficient Algorithms – much quicker than Asymmetric
 - Hard to break when using a large Key Size
 - Ideal for bulk encryption / decryption
- Weaknesses
 - Poor Key Distribution (must be done out of band – ie phone, mail, etc)
 - Poor Key Management / Scalability (each user needs a unique key)
 - Cannot provide authenticity or non-repudiation – only confidentiality

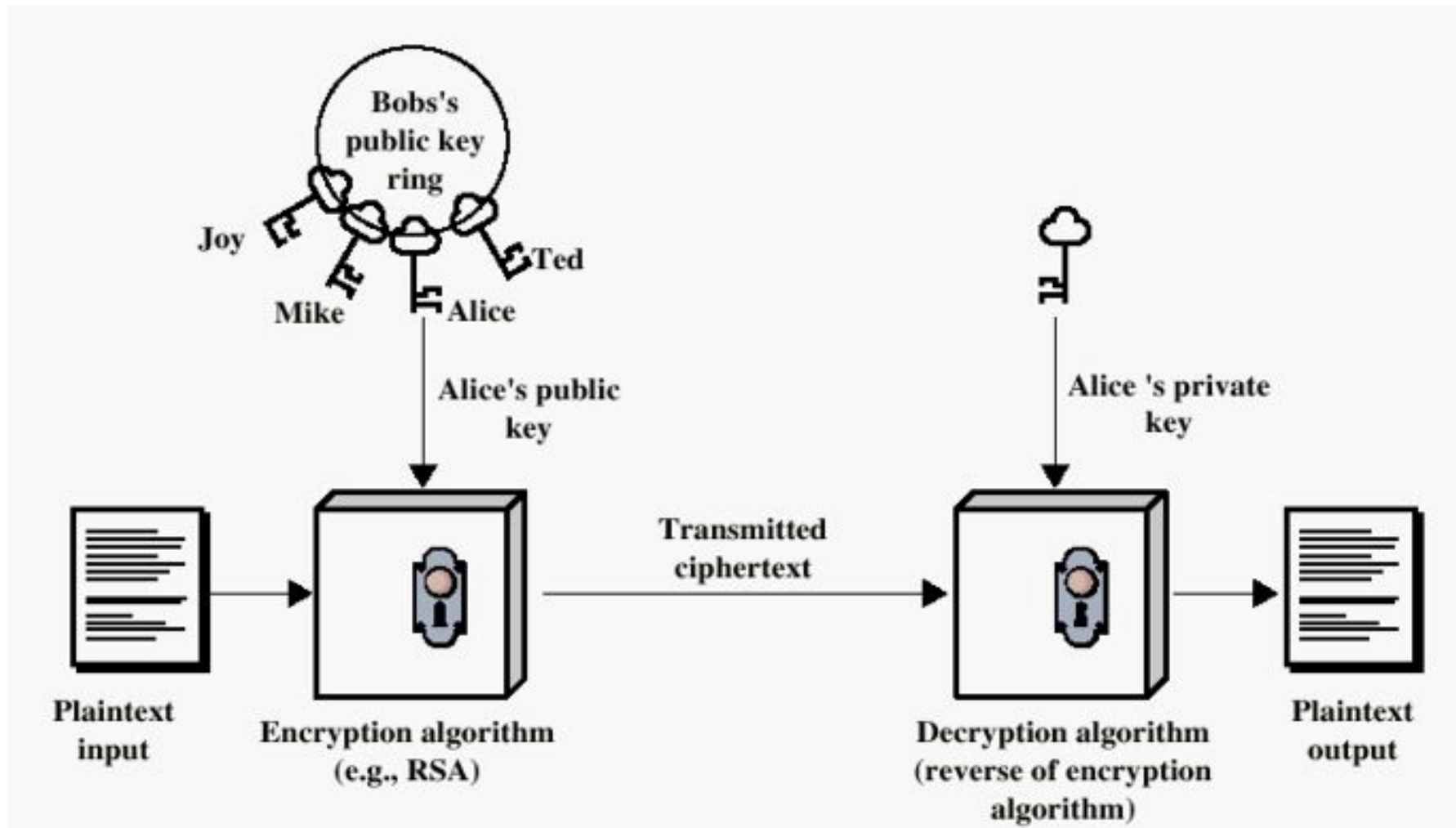
Why Public-Key Cryptography?

- Developed to address two issues:
 - **key distribution** – how to have secure communications in general without having to trust a KDC with your key
 - **digital signatures** – how to verify a message comes intact from the claimed sender
- Whitfield Diffie and Martin Hellman in 1976 known earlier in classified community

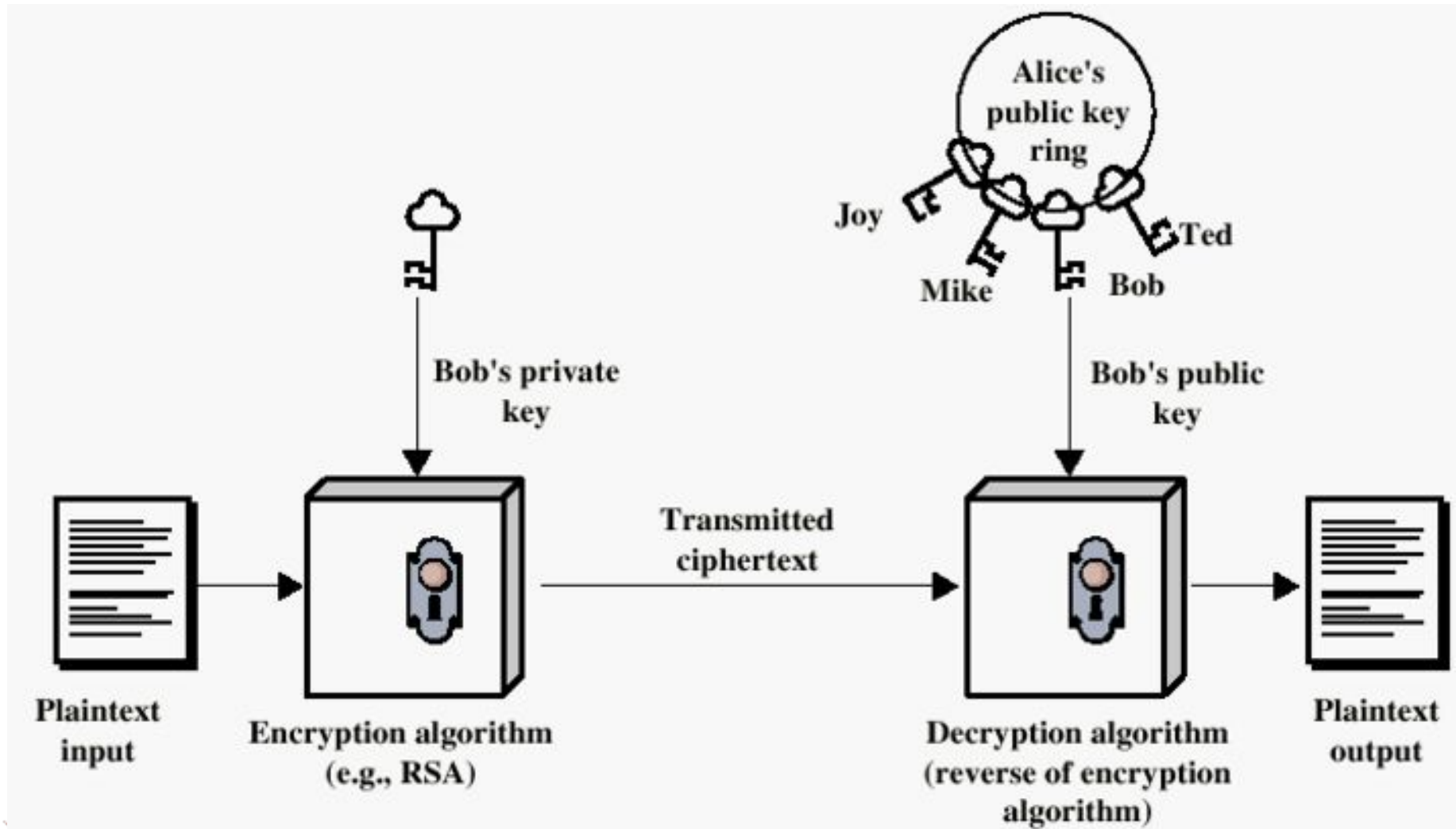
Public-Key Cryptography Principles

- # The use of two keys has consequences in: key distribution, confidentiality and authentication.
- # The scheme has six ingredients
 - ▣ Plaintext
 - ▣ Encryption algorithm
 - ▣ Public and private key
 - ▣ Ciphertext
 - ▣ Decryption algorithm

Encryption using Public-Key system



Authentication using Public-Key System



Applications for Public-Key Cryptosystems

✦ Three categories:

✦ **Encryption/decryption:** The sender encrypts a message with the recipient's public key.

✦ **Digital signature:** The sender "signs" a message with its private key.

✦ **Key exchange:** Two sides cooperate to exchange a session key.

Public-Key Cryptographic Algorithms

RSA - Ron Rives, Adi Shamir and Len Adleman at MIT, in 1977.

- ✚ RSA is a block cipher
- ✚ The most widely implemented

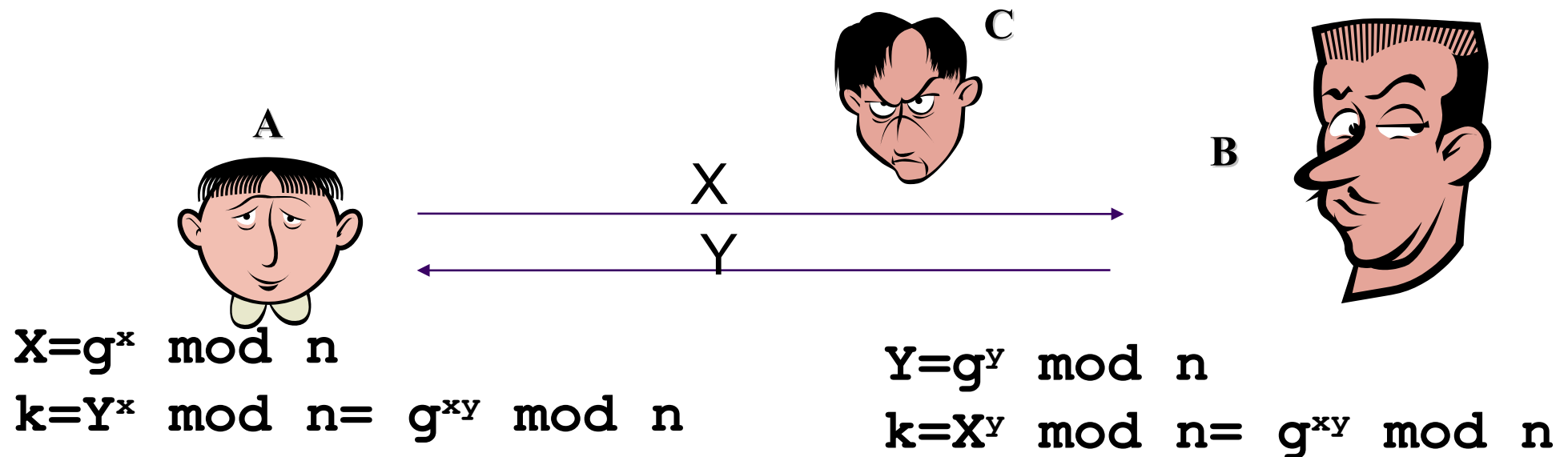
Diffie-Hellman

- ✚ Exchange a secret key securely
- ✚ Compute discrete logarithms

Elliptic Curve Cryptography (ECC)

Diffie-Hellman Key Agreement

- Published in 1976
- Based on difficulty of calculating discrete logarithm in a finite field
- Two parties agreed on two large numbers n and g , such that g is a prime with respect to n



Possible to do man in the middle attack

Diffie-Hellman Key Agreement

- Alice and Bob publicly agree to use a modulus **$n = 23$** and base **$g = 5$**
- Alice chooses a secret integer **$a = 3$** , then sends Bob $A = g^a \bmod p$
- **$A = 125 \bmod 23 = 10$**
- Bob chooses a secret integer **$b = 2$** , then sends Alice $B = g^b \bmod p$
- **$B = 25 \bmod 23 = 2$**
- Alice computes **$K = B^a \bmod p$**
- **$K = 8 \bmod 23 = 8$**
- Bob computes **$K = A^b \bmod p$**
- **$K = 100 \bmod 23 = 8$**
- Alice and Bob now share a secret (the number 8).

Revest-Shamir-Adelman (RSA)

By Rivest, Shamir and Adelman in 1978

1. Find 2 large prime numbers p and q (100 digits=512bits)
2. Calculate the product $n=p*q$ (n is around 200 digits)
3. Select large integer e relatively prime to $(p-1)(q-1)$
*Relatively prime means e has no factors in common with $(p-1)(q-1)$.
Easy way is select another prime that is larger than both $(p-1)$ and $(q-1)$.*
4. Select d such that $e*d \bmod (p-1)*(q-1)=1$

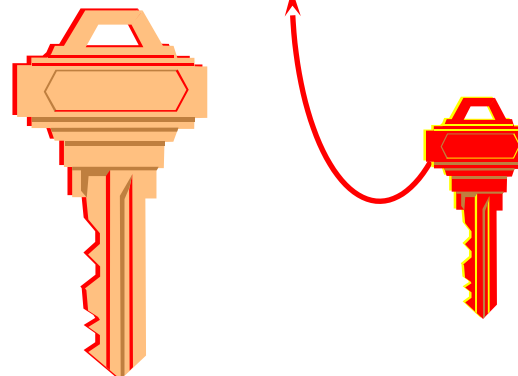
Encryption

$$C=P^e \bmod n$$

Decryption

$$P=C^d \bmod n$$

Two keys are d and e along with n



RSA - Simple Example

1. Find 2 prime numbers p and q

Let $p=11$ and $q=17$

2. Calculate the product $n=p*q$

$$n = 11*17=187$$

3. Select large integer e relatively prime to $(p-1)(q-1)$

*$E=7$; 7 IS Relatively prime to $(p-1)(q-1) = 10*16=160$*

4. Select d such that $e*d \bmod (p-1)*(q-1)=1$

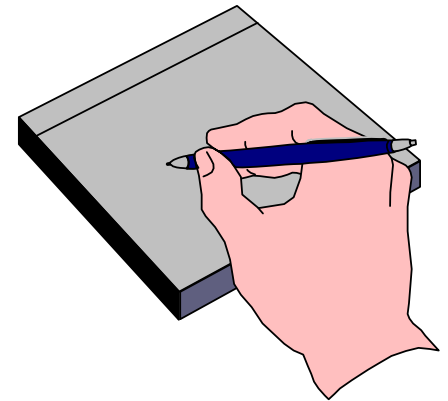
*$d=23$ because, $23*7 \bmod 10*16=161 \bmod 160 =1$*

Encryption

$$C=P^e \bmod n$$

Decryption

$$P=C^d \bmod n$$



RSA - Simple Example

recipient knows:

- $PR=\{23,187\}$ // $d=23, n=187$
- $187=17 \times 11$ // $p=17, q=11$
- $\phi(n)=(p-1)(q-1)=160$ // check: $7 \times 23 \bmod 160=1$

sender knows:

- $PU=\{7,187\}$ // $e=7, n=187$
- plaintext to encrypt: $M=88$ // $88 < 187$

RSA - Simple Example

sender knows:

- $PU = \{7, 187\}$
- plaintext to encrypt: $M = 88$ // $88 < 187$

Encryption

$$88^7 \bmod 187 = [(88^4 \bmod 187) \times (88^2 \bmod 187) \times (88^1 \bmod 187)] \bmod 187$$

$$88^1 \bmod 187 = 88$$

$$88^2 \bmod 187 = 7744 \bmod 187 = 77$$

$$88^4 \bmod 187 = 59,969,536 \bmod 187 = 132$$

$$88^7 \bmod 187 = (88 \times 77 \times 132) \bmod 187 = 894,432 \bmod 187 = 11$$

ciphertext



RSA - Simple Example

recipient knows:

- $PR = \{23, 187\}$
- $187 = 17 \times 11$ // $p=17, q=11$
- $\phi(n) = (p-1)(q-1) = 160$ // check: $7 \times 23 \bmod 160 = 1$
- receives cipher text: 11

Decryption

$$11^{23} \bmod 187 = [(11^1 \bmod 187) \times (11^2 \bmod 187) \times (11^4 \bmod 187) \times (11^8 \bmod 187) \times (11^8 \bmod 187)] \bmod 187$$

$$11^1 \bmod 187 = 11$$

$$11^2 \bmod 187 = 121$$

$$11^4 \bmod 187 = 14,641 \bmod 187 = 55$$

$$11^8 \bmod 187 = 214,358,881 \bmod 187 = 33$$

$$\begin{aligned} 11^{23} \bmod 187 &= (11 \times 121 \times 55 \times 33 \times 33) \bmod 187 \\ &= 79,720,245 \bmod 187 = 88 \end{aligned}$$

plaintext

RSA --- 2nd small example (1)

- Kamal:
 - ✚ chooses 2 primes: $p=5, q=11$
multiplies p and q : $n = p * q = 55$
 - ✚ finds out two numbers $e=3$ & $d=27$ which satisfy
 $(3 * 27) \bmod 40 = 1$
 - ✚ Kamal's public key
 - 2 numbers: $(3, 55)$
 - encryption alg: modular exponentiation
 - ✚ secret key: $(27, 55)$

RSA --- 2nd small example (2)

- Amal has a message $m=13$ to be sent to Kamal:
 - ✚ finds out Kamal's public encryption key $(3, 55)$
 - ✚ calculates c :
$$\begin{aligned}c &= m^e \pmod n \\&= 13^3 \pmod{55} \\&= 2197 \pmod{55} \\&= 52\end{aligned}$$
 - ✚ sends the ciphertext $c=52$ to Kamal

RSA --- 2nd small example (3)

- Kamal:
 - † receives the ciphertext $c=52$ from Amal
 - † uses his matching secret decryption key 27 to calculate m :
$$m = 52^{27} \pmod{55}$$
$$= 13 \text{ (Amal's message)}$$

RSA Signature --- an eg (1)

- Kamal:
 - ✚ chooses 2 primes: $p=5, q=11$
multiplies p and q : $n = p * q = 55$
 - ✚ finds out two numbers $e=3$ & $d=27$ which satisfy
 $(3 * 27) \bmod 40 = 1$
 - ✚ Kamal's public key
 - 2 numbers: $(3, 55)$
 - encryption algo: modular exponentiation
 - ✚ secret key: $(27, 55)$

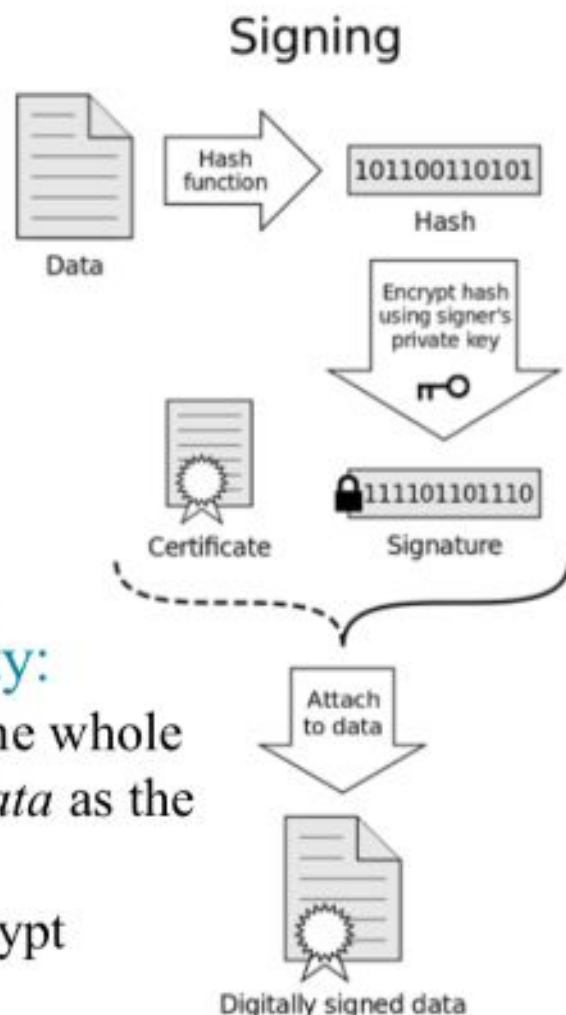
RSA Signature --- an eg (2)

- Kamal has a document $m=19$ to sign:
 - † uses his secret key $d=27$ to calculate the digital signature of $m=19$:
$$\begin{aligned}s &= m^d \pmod n \\ &= 19^{27} \pmod{55} \\ &= 24\end{aligned}$$
 - † appends 24 to 19. Now $(m, s) = (19, 24)$ indicates that the doc is 19, and Kamal's signature on the doc is 24.

RSA Signature --- an eg. (3)

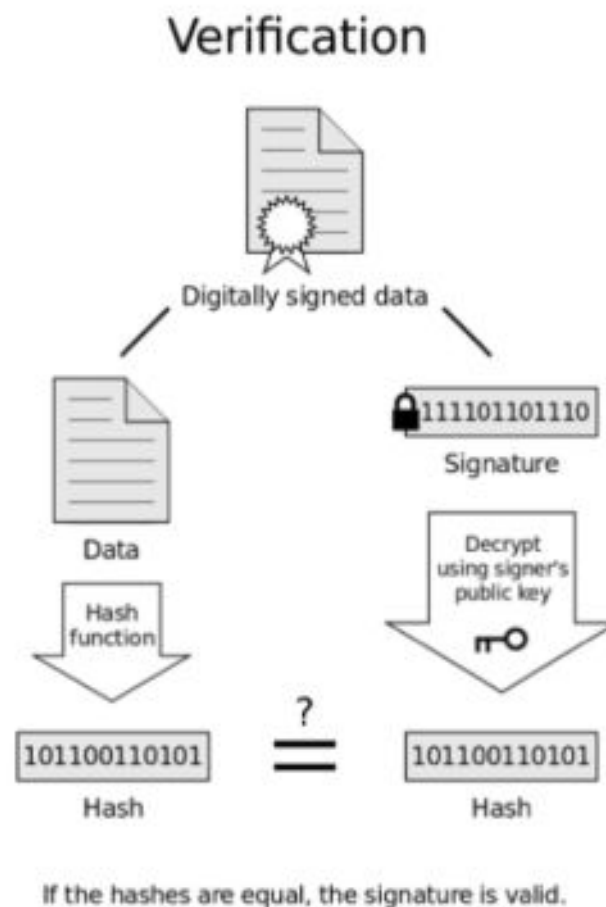
- Nimal, a verifier:
 - † receives a pair $(m,s)=(19, 24)$
 - † looks up the phone book and finds out Kamal's public key $(e, n)=(3, 55)$
 - † calculates
$$\begin{aligned} t &= s^e \pmod n \\ &= 24^3 \pmod{55} \\ &= 19 \end{aligned}$$
 - † checks whether $t=m$
 - † confirms that $(19,24)$ is a genuinely signed document of Kamal if $t=m$.

Typical Digital Signature



For **confidentiality**:

- Need to encrypt the whole *digitally signed data* as the plaintext.
- Four encrypt/decrypt operations!



Signature Creation

- **Generate Public/Private key pair**

```
openssl genrsa -out mykey.pem
```

```
openssl rsa -in mykey.pem -pubout >mypub.pem
```

- **Create the signature**

```
openssl dgst -sha1 -sign mykey.pem  
-out mysign.sha1 jethavanaya.jpg
```



Signature Verification

- **Retrieves the Public key**

- **Verify the signature**

```
openssl dgst -sha1 -verify mypub.pem  
-signature mysign.sha1 jethavanaya.jpg
```



Elliptic curve cryptography (ECC)

ECC invented (independently):

- 1985
 - wide-scale adoption circa 2005
- barrier to adoption: patent/license protections



Neal Koblitz
born in 1948



Victor S. Miller
born in 1947

Elliptic curve cryptography (ECC)

Elliptic curves have been studied by mathematicians for over a hundred years. They have been deployed in diverse areas

- Number theory: proving Fermat's Last Theorem in 1995 [4]
 - ✚ The equation $x^n + y^n = z^n$ has no nonzero integer solutions for x, y, z when the integer n is greater than 2.
- Modern physics: String theory
 - ✚ The notion of a point-like particle is replaced by a curve-like string.
- Elliptic Curve Cryptography
 - ✚ An efficient public key cryptographic system.

Elliptic curve cryptography (ECC)

Elliptic curves over real numbers

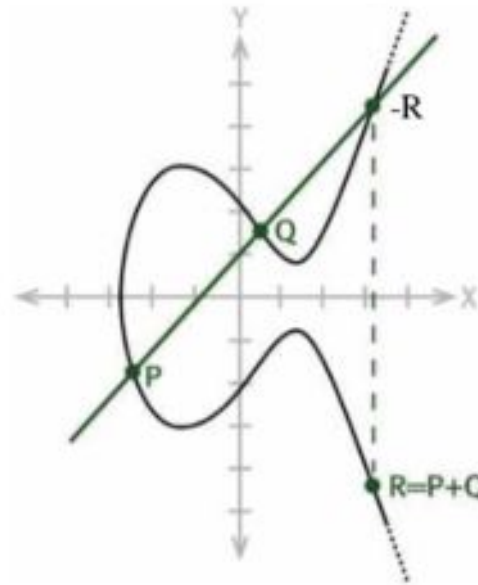
- Calculations prove to be slow
- Inaccurate due to rounding error
- Infinite field

Cryptographic schemes need fast and accurate arithmetic

- In the cryptographic schemes, elliptic curves over two finite fields are mostly used.
 - Prime field $\mathbb{F}_p$, where p is a prime.
 - Binary field $\mathbb{F}_{2^m}$, where m is a positive integer.

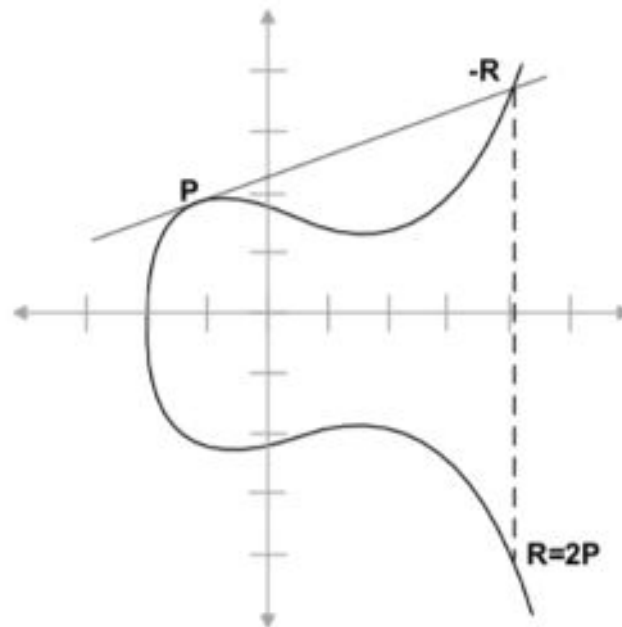
Point Addition

To add two distinct points P and Q on an elliptic curve, draw a straight line between them. The line will intersect the elliptic curve at exactly one more point $-R$. The reflection of the point $-R$ with respect to x -axis gives the point R , which is the results of addition of points P and Q .



Point Doubling

To the point P on elliptic curve, draw the tangent line to the elliptic curve at P . The line intersects the elliptic curve at the point $-R$. The reflection of the point $-R$ with respect to x -axis gives the point R , which is the results of doubling of point P .



ECC Cryptography

Elliptic curves are used to construct the public key cryptography system

The private key d is randomly selected from $[1, n-1]$, where n is integer.

Then the public key Q is computed by dP , where P, Q are points on the elliptic curve.

Like the conventional cryptosystems, once the key pair (d, Q) is generated, a variety of cryptosystems such as signature, encryption/decryption, key management system can be set up.

Computing dP is denoted as scalar multiplication. It is not only used for the computation of the public key but also for the signature, encryption, and key agreement in the ECC system.

Scalar Multiplication

Intuitive approach:

$$dP = \underbrace{P + P + \dots + P}_{d \text{ times}}$$

It requires $d-1$ times point addition over the elliptic curve.

Observation: To compute $17P$, we could start with $2P$, double that, and that two more times, finally add P , i.e. $17P = 2(2(2(2P))) + P$. This needs only 4 point doublings and one point addition instead of 16 point additions in the intuitive approach. This is called Double-and-Add algorithm.

Elliptic Curve Digital Signature Algorithm (ECDSA)

Alice

Private key d_A , Public key $Q_A = d_A P$.

Signature generation

1. Select a random k from $[1, n-1]$
2. Compute $kP = (x_1, y_1)$ and $r = x_1 \bmod n$. if $r=0$ goto step 1
3. Compute $e = H(m)$, where H is a hash function, m is the message.
4. Compute $s = k^{-1}(e + d_A r) \bmod n$. If $s=0$ go to step 1.

(r, s) is Alice's signature of message m

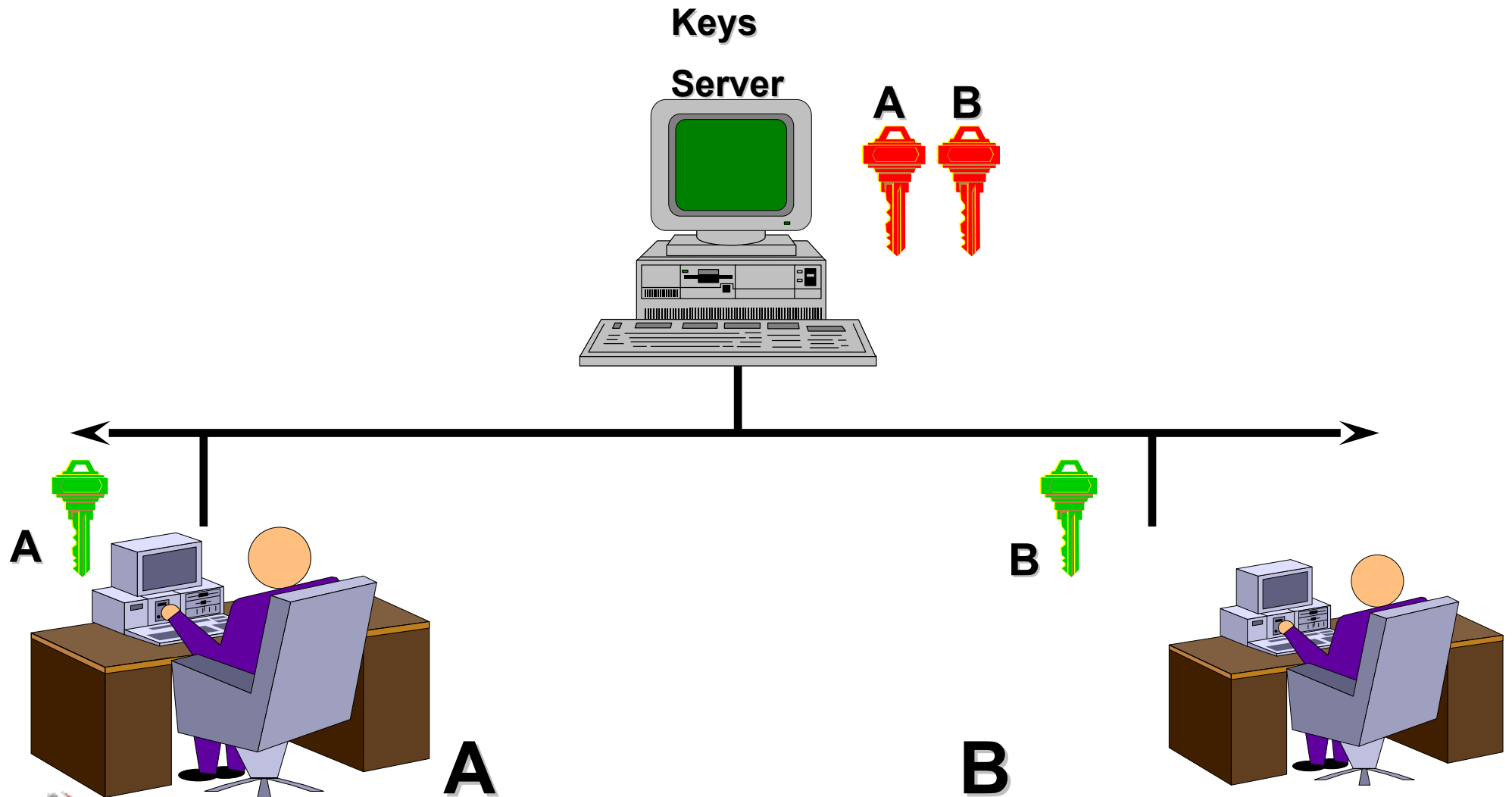
$m, (r, s)$

Bob

Signature verification

1. Verify that r, s are in the interval $[1, n-1]$
2. Compute $e = H(m)$, where H is a hash function, m is the message.
3. Compute $w = s^{-1} \bmod n$
4. Compute $u_1 = ew \bmod n$ and $u_2 = rw \bmod n$.
5. Compute $X = u_1 P + u_2 Q_A = (x_1, y_1)$
6. Compute $v = x_1 \bmod n$
7. Accept the signature if and only if $v = r$

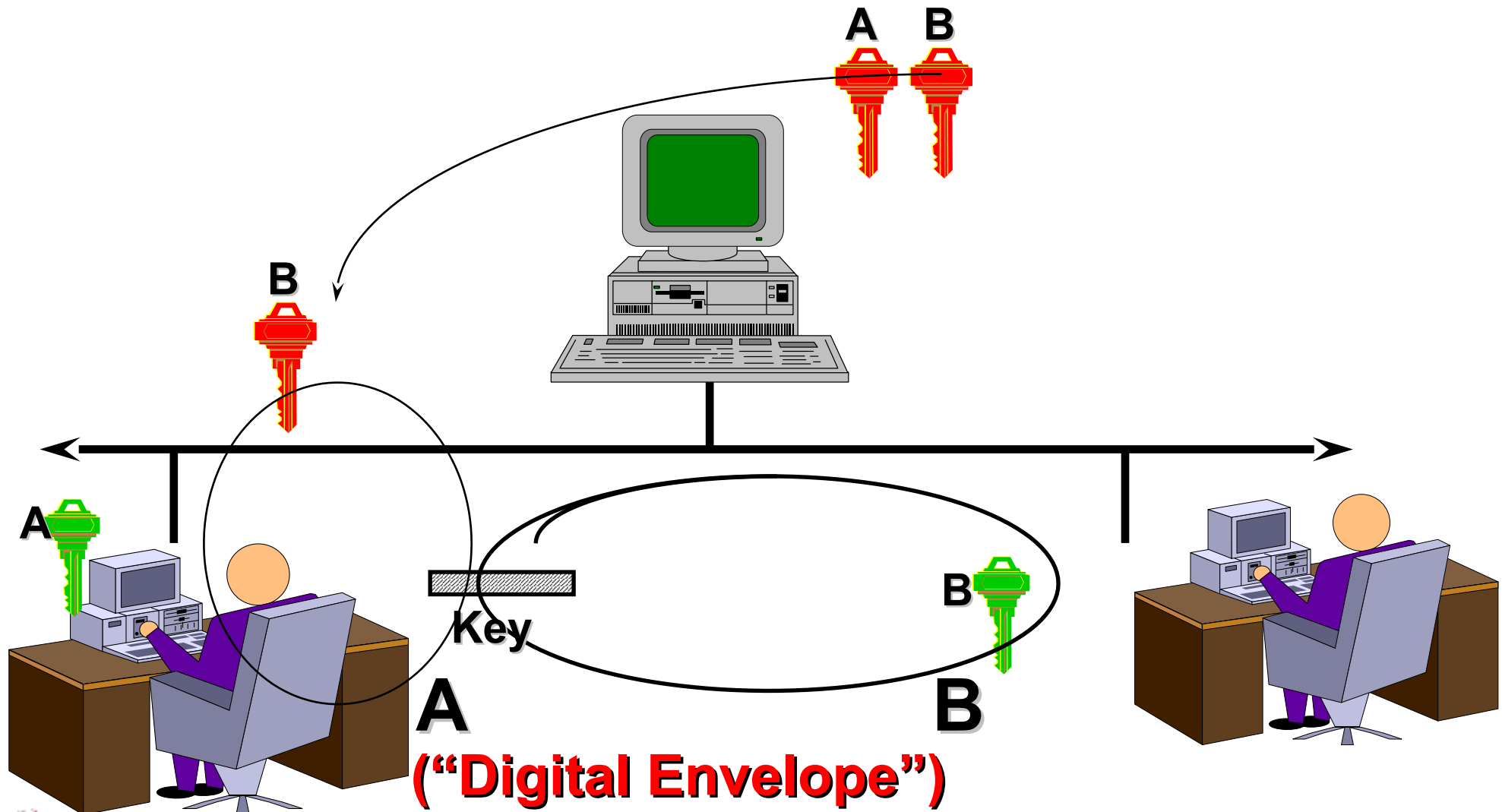
Storage and Handling Public Keys



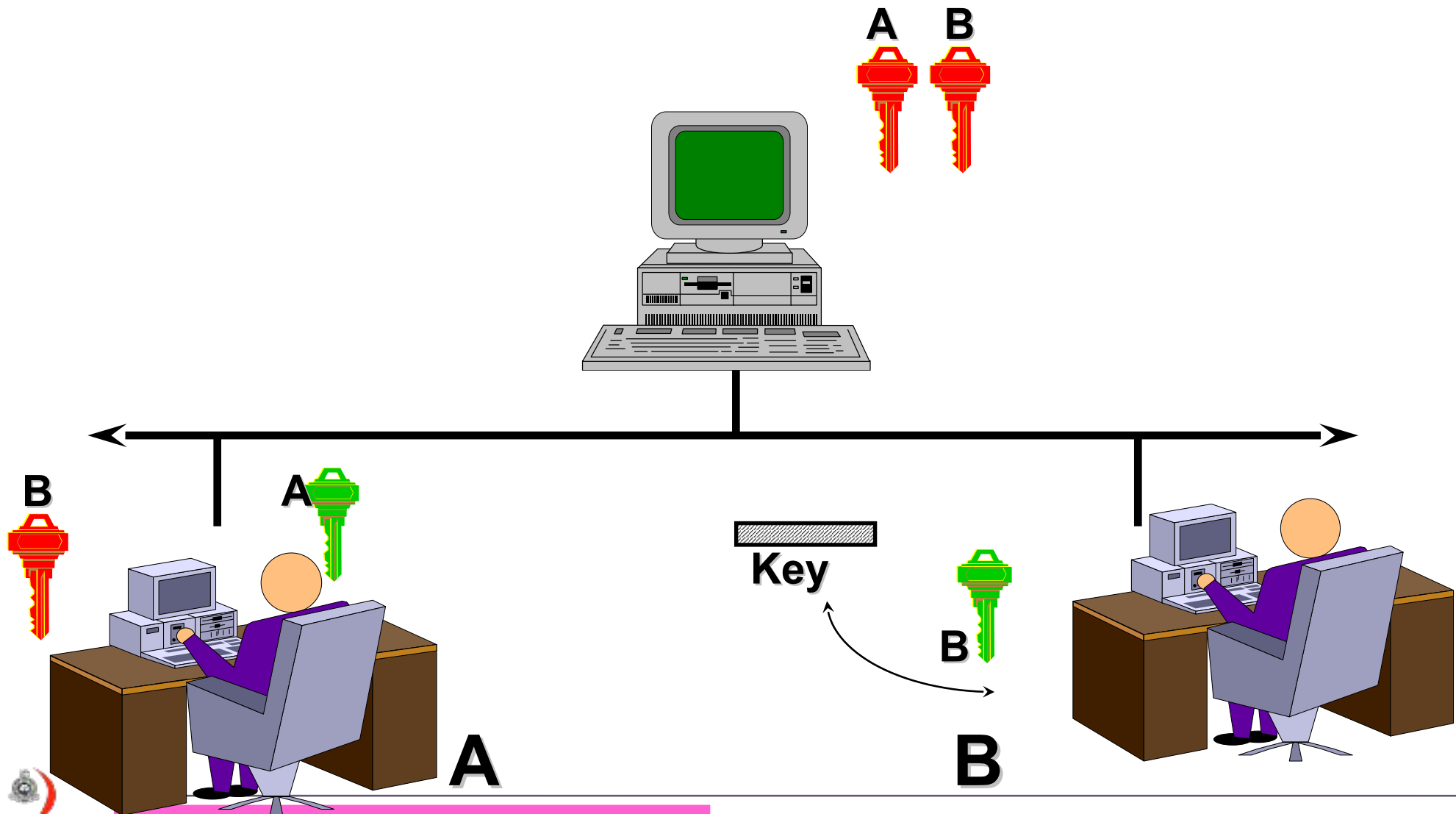
Key Management

- Using a public key system, A wants to talk to B
- C is the Key Distribution Center(Key Server), has A and B's public key
- A calls B, and the calling protocol contacts C
- C encrypts a session key, "k", with the public keys and sends the encrypted "k" to A and B
- A and B can then communicate

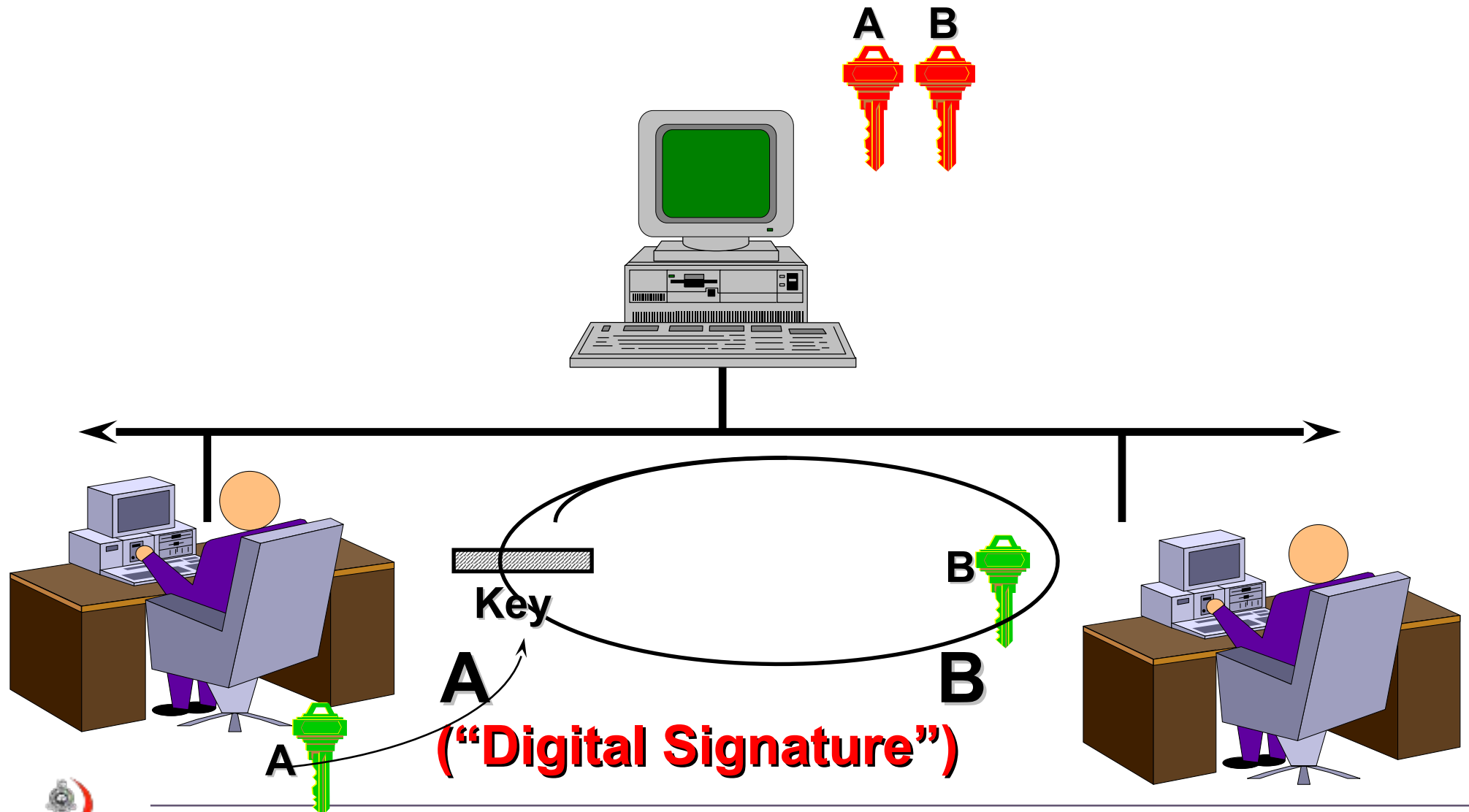
Secure Sending of secret key



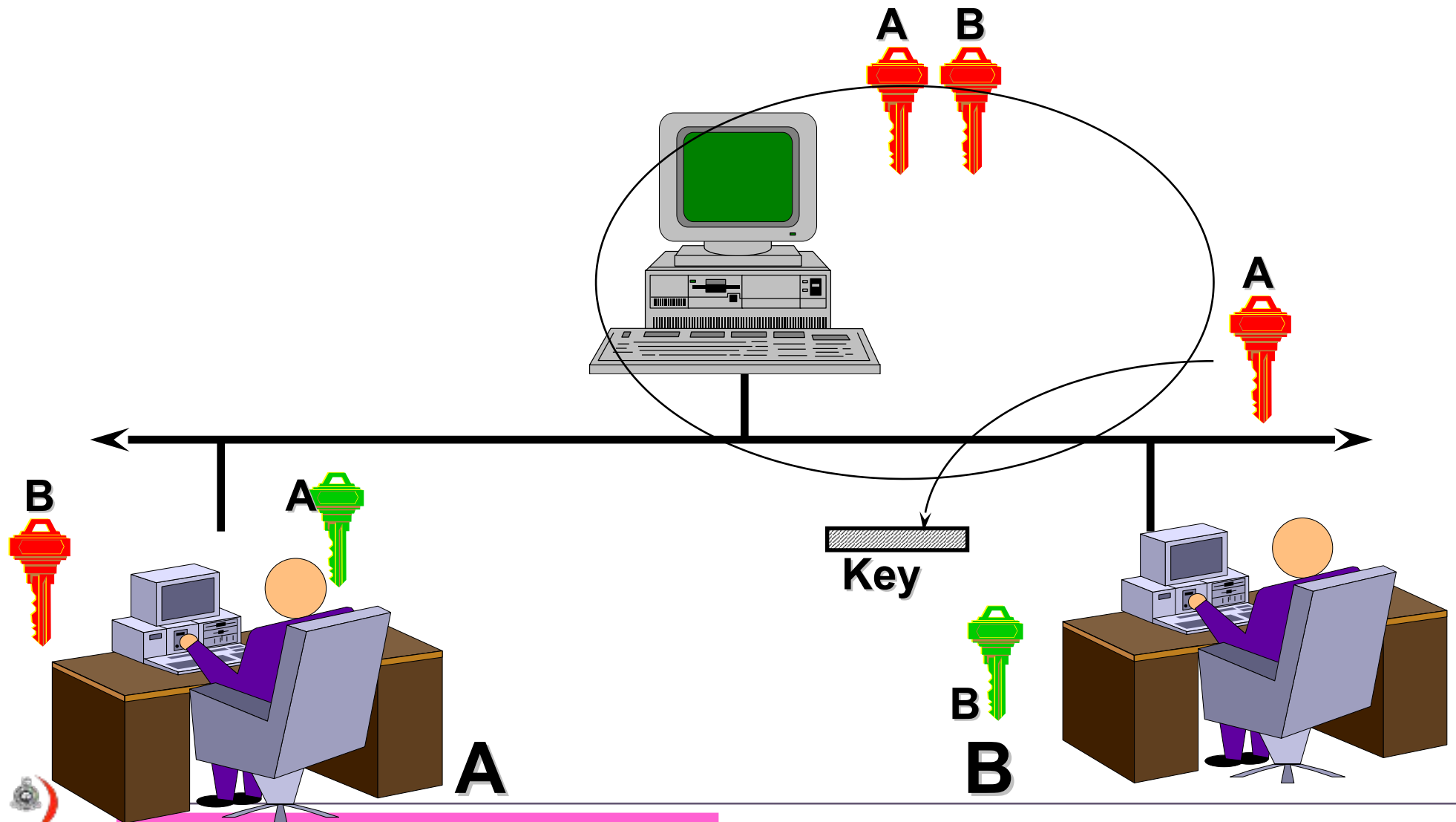
Recovery of Secret Key



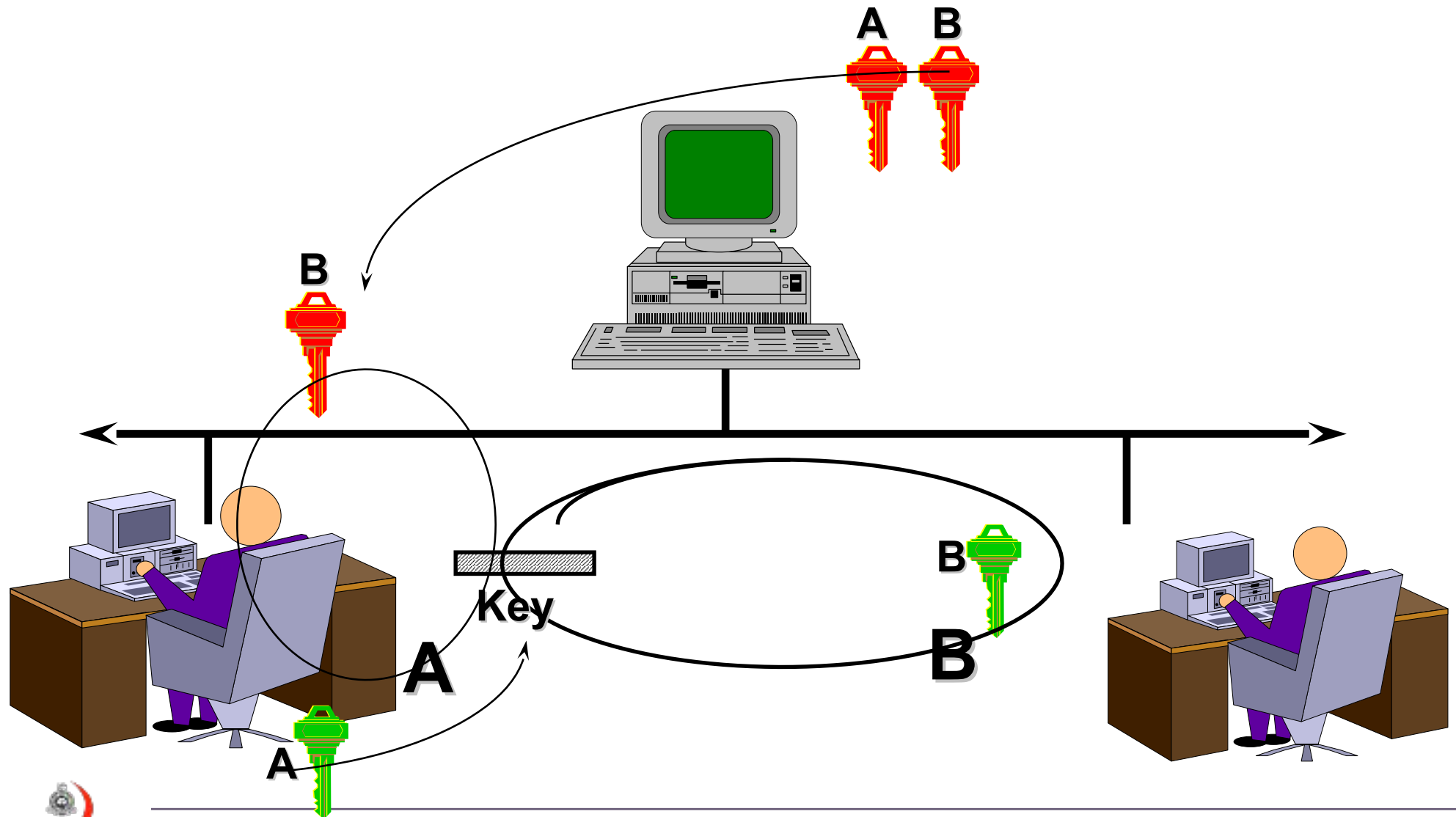
Authenticity of Sender



Verification of Signature

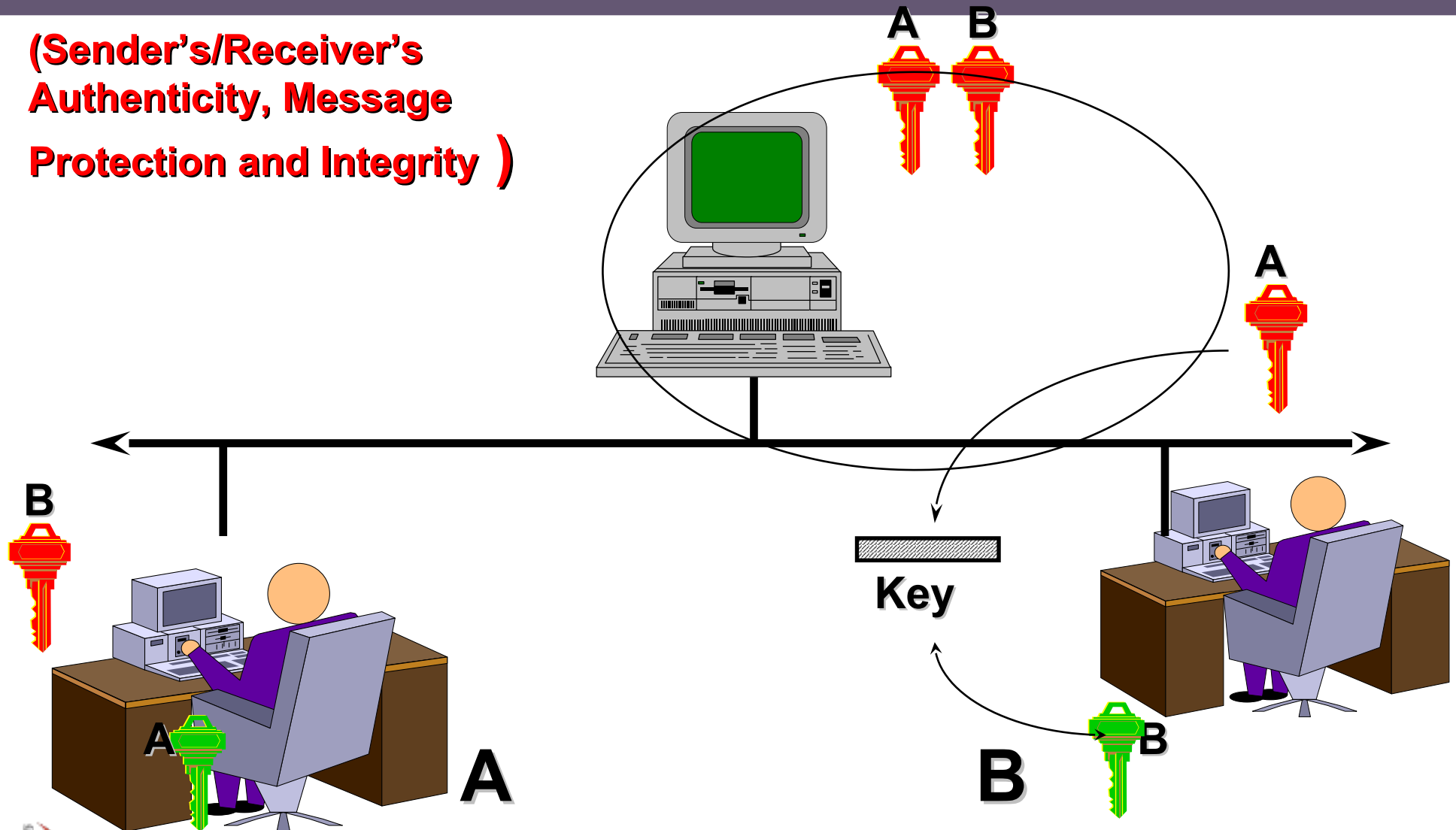


Authenticity of Sender and Receiver

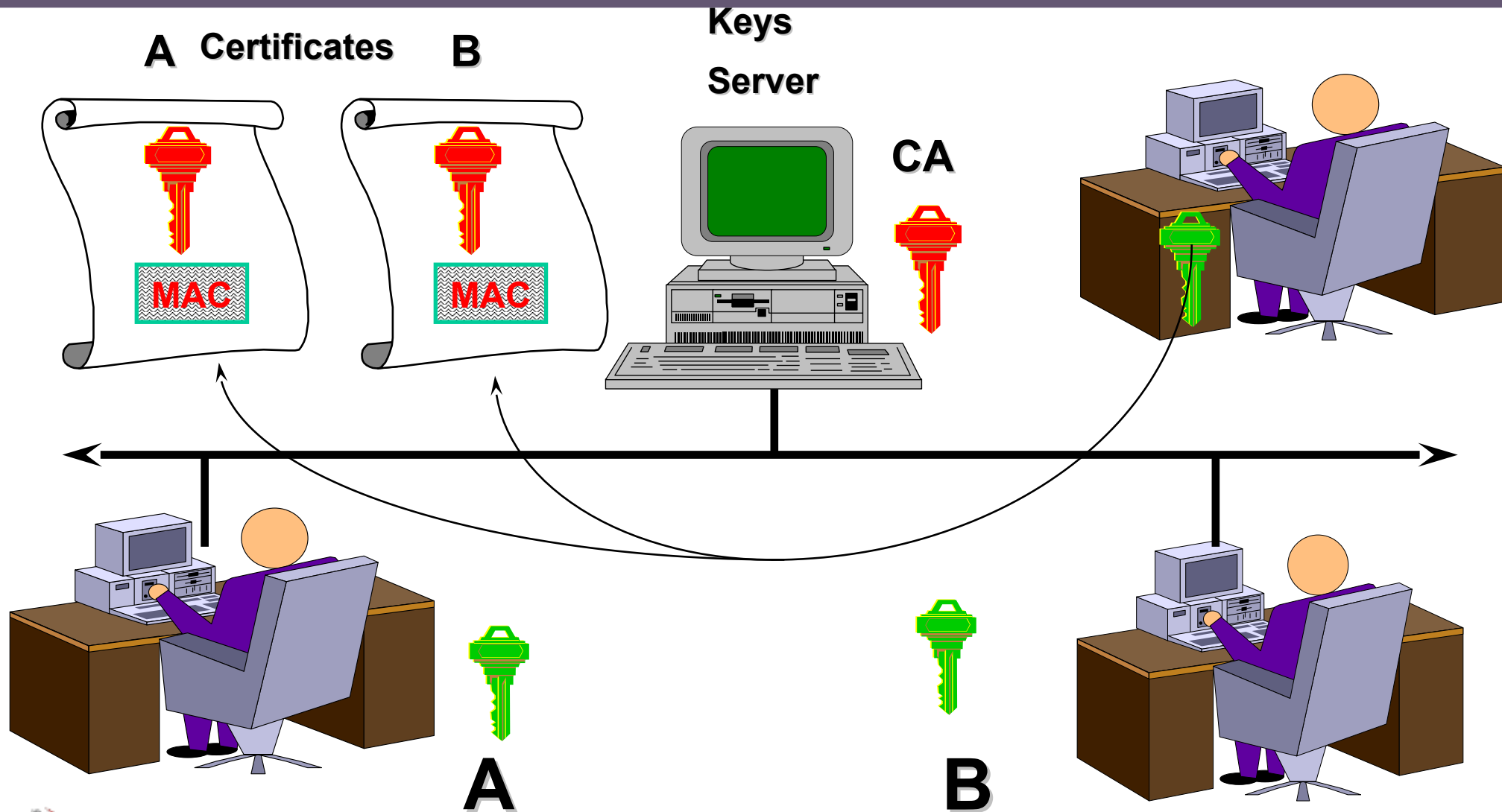


Full Verification

**(Sender's/Receiver's
Authenticity, Message
Protection and Integrity)**



Certificate Authority



Certificates Infrastructure

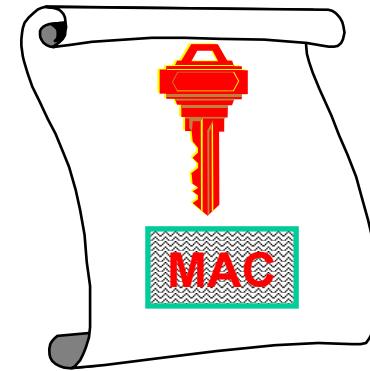
- Certificates need some infrastructure in place to allow users to verify a given certificate.
- This can be done centrally or via a distributed system.
- So how are certificates, and their certificate chains, verified and disseminated?
 - (1) Trusted Third Party (TTP)
 - (2) Certificate Authority (CA)
 - (3) Simple Public Key Infrastructure (SPKI)

Certificates Infrastructure

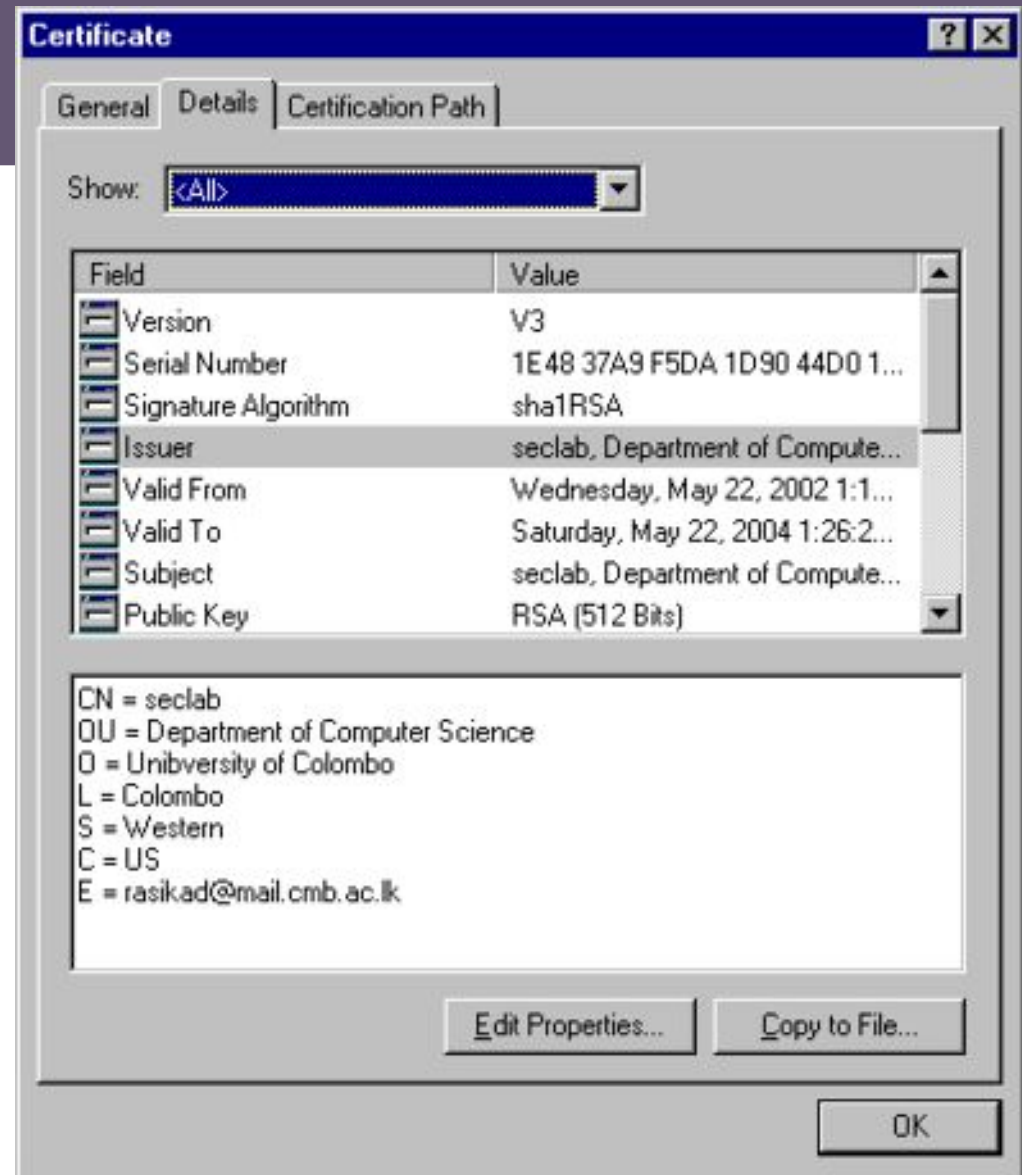
- Trusted, 3rd party organization
- CA (Certificate Authority) guarantees that the individual granted a certificate is who he/she claims to be
- CA usually has arrangement with financial institution to confirm identity
- Critical to data security and electronic commerce
- Well known organisation establish themselves to act as certificate authorities. Verisign, CREN, etc.
- One can then obtain X.509 public key certificates from them by submitting satisfactory evidence of their identity.

Internal Structure of Certificate

- Version
- Serial Number
- Signature Algorithm
- Issuer
- Subject
- Validity
- Subject Public Key Information
- Extensions
- Signature



Root Certificate



Public key infrastructure (PKI)

- Public key infrastructure (PKI) - provides the foundation necessary for secure e-business through the use of cryptographic keys and certificates
 - Enables secure electronic transactions
 - Enables the exchange of sensitive information

PKI

Public Key Infrastructure (PKI)

- PKI is an ISO authentication “framework” that uses public key cryptography and X.509 standard protocols.
- The framework establishes a generalized architecture for exchanging secure communication across networks. (Internet, internal / external).
- PKI is a Hybrid Key System with an infrastructure allowing the PKI certificate authority to create, maintain and manage digital certificates.
- Each user is issued a Digital Certificate (DC) which contains the end users public key along with other identifying information.
- The Digital Certificate is signed / validated by a trusted third party / Certificate Authority (CA). The CA enables users who are not trusted to each other (unknown) to trust each other.
- The most popular DC is an X.509 v3 This is same type of certificate as an SSL / HTTPS Certificate.

Certificate Hierarchy

CA



CA



CA



A

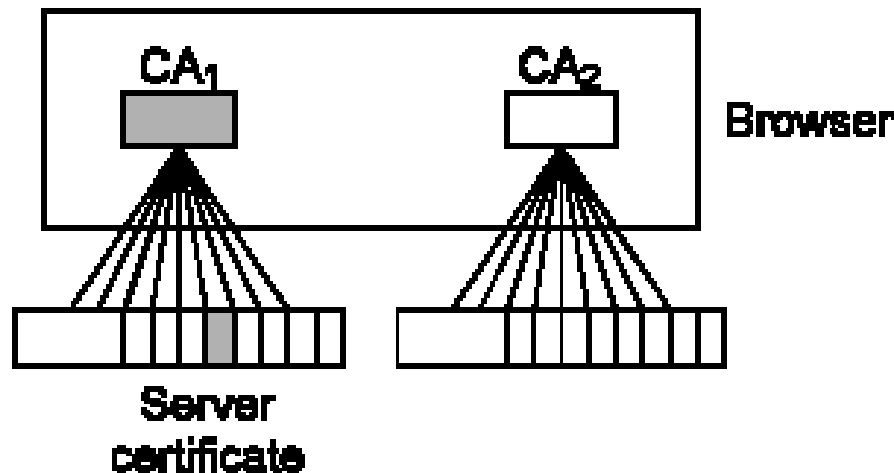


B



CA Hierarchy in Practice

Flat or Clayton's hierarchy

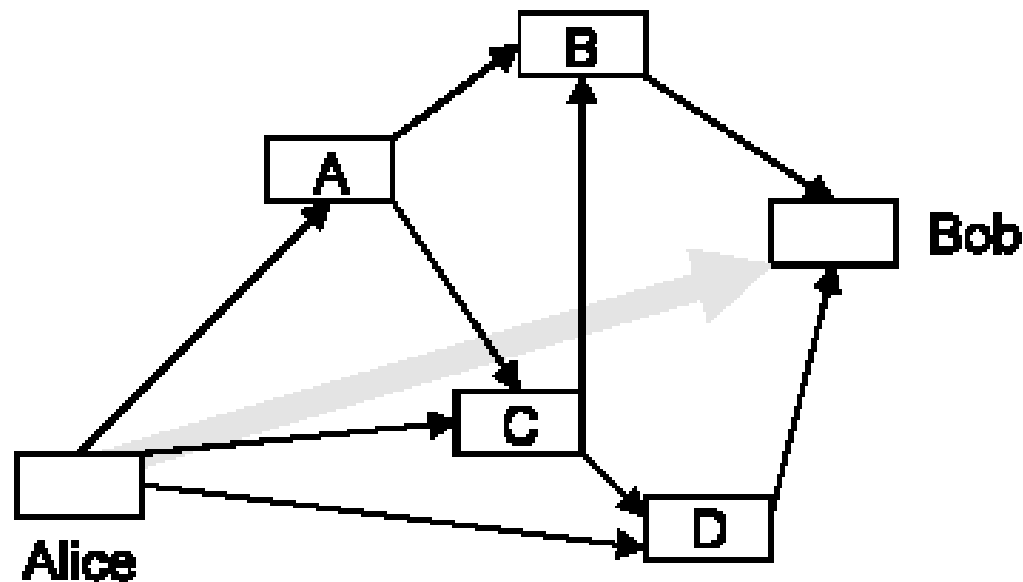


CA certificates are hard-coded into web browsers or email software

- Later software added the ability to add new CAs to the hardcoded initial set

Alternative Trust Hierarchies

PGP web of trust



Bob knows B and D who know A and C who know Alice
 $\Rightarrow$ Bob knows the key came from Alice

Web of trust more closely reflects real-life trust models

Certificate Revocation

- Revocation is managed with a Certificate Revocation List (CRL), a form of anti-certificate which cancels a certificate
- Equivalent to 1970s-era credit card blacklist booklets
- Relying parties are expected to check CRLs before using a certificate
- *“This certificate is valid unless you hear somewhere that it isn’t”*



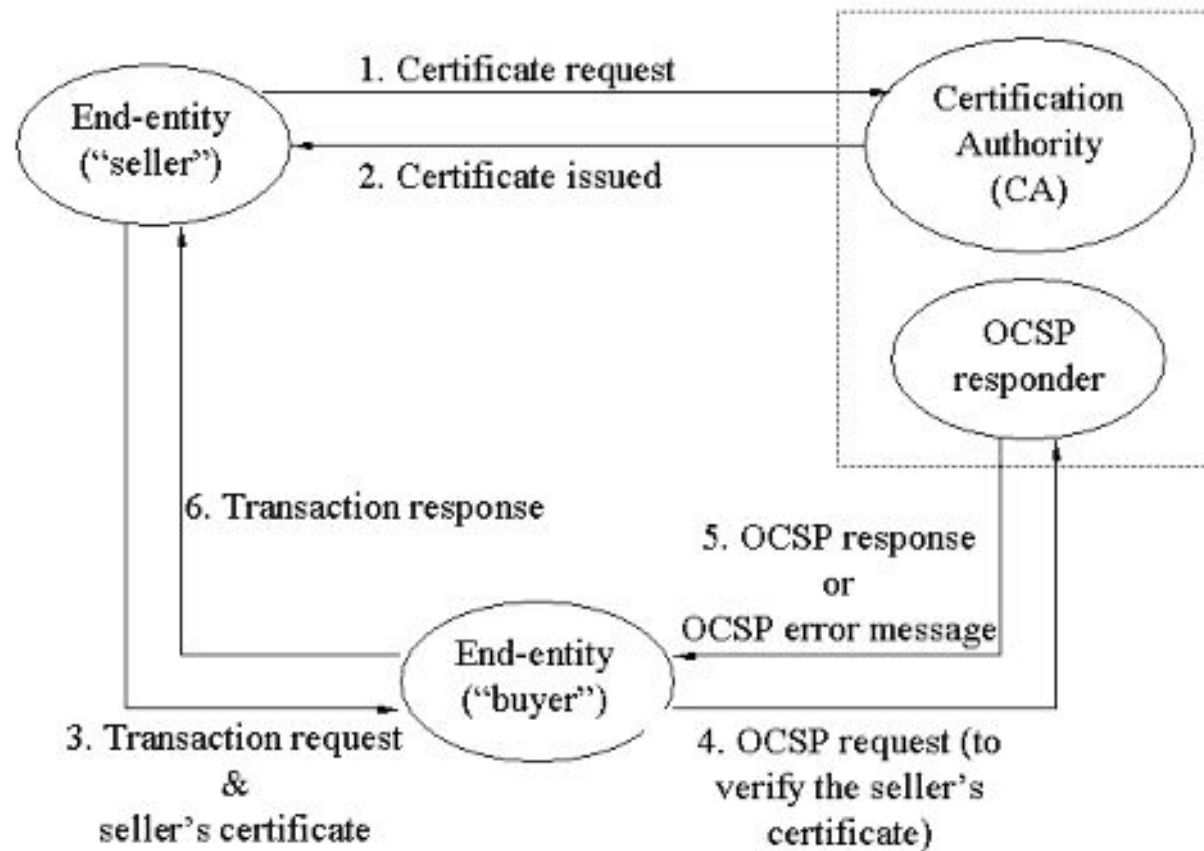
CRL Distribution Problems

- CRLs have a fixed validity period
 - Valid from *issue date* to *expiry date*
- At *expiry date*, all relying parties connect to the CA to fetch the new CRL
 - Massive peak loads when a CRL expires (DDOS attack)
- Issuing CRLs to provide timely revocation exacerbates the problem
 - 10M clients download a 1MB CRL issued once a minute = ~150GB/s traffic
 - Even per-minute CRLs aren't timely enough for high-value transactions with interest calculated by the minute

Online Status Checking

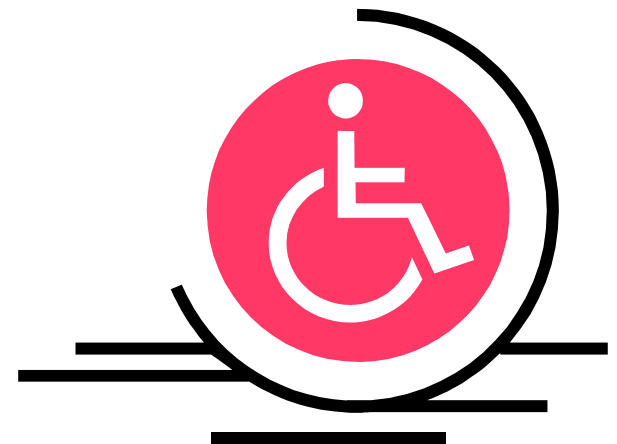
- Online Certificate Status Protocol, **OCSP**
- Inquires of the issuing CA whether a given certificate is still valid
 - Acts as a simple responder for querying CRL's
 - Still requires the use of a CRL to check validity
- OCSP acts as a selective CRL protocol
 - Standard CRL process: "Send me a CRL for everything you've got"
 - OCSP process: "Send me a pseudo-CRL/OCSP response for only these certs"
 - Lightweight pseudo-CRL avoids CRL size problems
 - Reply is created on the spot in response to the request
 - Ephemeral pseudo-CRL avoids CRL validity period problems

Online Status Checking



Online Certificate Status Protocol (OCSP)

- Returned status values are non-orthogonal
 - Status = “good”, “revoked”, or “unknown”
 - “Not revoked” doesn’t necessarily mean “good”
 - “Unknown” could be anything from “Certificate was never issued” to “It was issued but I can’t find a CRL for it”



OCSP Problems

- Problems are due in some extent to the CRL-based origins of OCSP
 - CRL can only report a negative result
 - “Not revoked” doesn’t mean a cert was ever issued
 - Some OCSP implementations will report “I can’t find a CRL” as “Good”
 - Some relying party implementations will assume “revoked” “not good”, so any other status = “good”
 - Much debate among implementors about OCSP semantics

Key measure: Encryption strength

The mathematic background of ECC is more complex than other cryptographic systems

- Geometry, abstract algebra, number theory

ECC provides greater security and more efficient performance than the first generation public key techniques (RSA and Diffie-Hellman)

- Mobile systems
- Systems required high security level (such as 256 bit AES)

Bits of Security	Symmetric Key Algorithm	Corresponding RSA Key Size	Corresponding ECC Key Size
80	Triple DES (2 keys)	1024	160
112	Triple DES (3 keys)	2048	224
128	AES-128	3072	256
192	AES-192	7680	384
256	AES-256	15360	512

Factoring a product of two large primes

The best known conventional algorithm requires the solution time proportional to:

$$T(n) = \exp \left[c (\ln n)^{1/3} (\ln \ln n)^{2/3} \right]$$

For p & q 65 digits long T(n) is approximately one month using cluster of workstations.

For p&q 200 digits long T(n) is astronomical.

Quantum Computing algorithm for factoring.

- # **In 1994 Peter Shor from the AT&T Bell Laboratory showed that in principle a quantum computer could factor a very long product of primes in seconds.**
- # **Shor's algorithm time computational complexity is**

$$T(n) = O[(\ln n)^3]$$

Once a quantum computer is built the RSA method would not be safe.

Quantum Computing



Superconducting Qubits built using Josephson junctions.
Operate at ~15 millikelvin inside a dilution refrigerator (colder than outer space).
Controlled by microwave pulses that rotate qubits on the Bloch sphere.
IBM Condor (2023): 1121 qubits
Qiskit (open-source Python SDK).

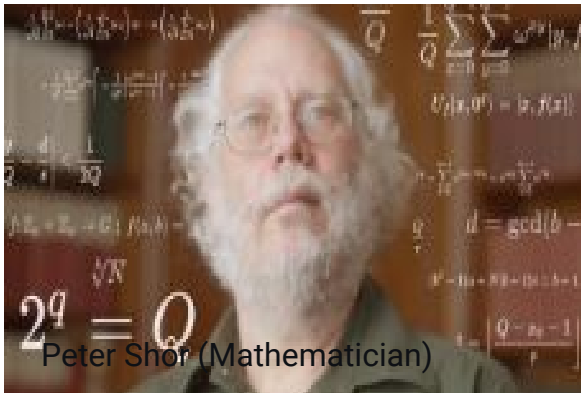


Superconducting qubits, arranged in a 2D grid.
Uses tunable couplers for precise control and reduced cross-talk.
Operated at cryogenic temperatures similar to IBM's systems.
Cirq-Python framework



Pursuing topological qubits based on Majorana chip
Hardware still in the experimental research phase.
QDK (Quantum Development Kit) supports Q#, Python, and .NET languages

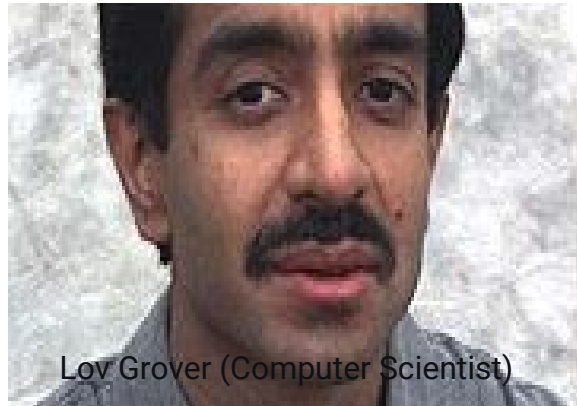
Quantum Algorithms



Peter Shor (Mathematician)

Solves the factorization in time that grows polynomially with the number of bits.

This would make RSA encryption practically breakable on a large, fault-tolerant quantum computer.



Lov Grover (Computer Scientist)

Search an unstructured database of N items for one unique marked item.

Searching 1 correct password out of 1,000,000

Classical: ~500,000 tries

Quantum: ~1,000 tries.



Quantum analog of the Discrete Fourier Transform (DFT). Transforms computational basis states into phase-encoded superpositions.

Core of Shor's Algorithm (for period finding).

The Foundation of Modern Encryption

The Foundation of Modern Encryption:

"These problems are too hard for classical computers"

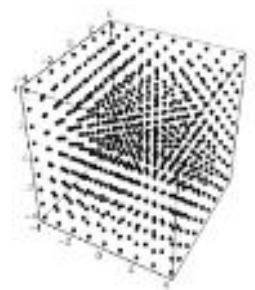
Factoring large numbers (RSA)

Discrete logarithm problems (ECC)

This promise has held for 40+ years

This promise is about to be broken

"Now we need problems too hard for quantum computers as well"



Post-Quantum Cryptography (PQC)

Algorithms that run on **classical computers** but are **mathematically resistant** to attacks from **quantum computers**.

The security is based on the perceived computational difficulty of solving the **Shortest Vector Problem (SVP)**, the **Short Integer Solution (SIS)** or the **Learning With Errors (LWE)** problem in **high-dimensional lattices**.



The U.S. National Institute of Standards and Technology (NIST) is finalizing standards.

Post-Quantum Cryptography (PQC)

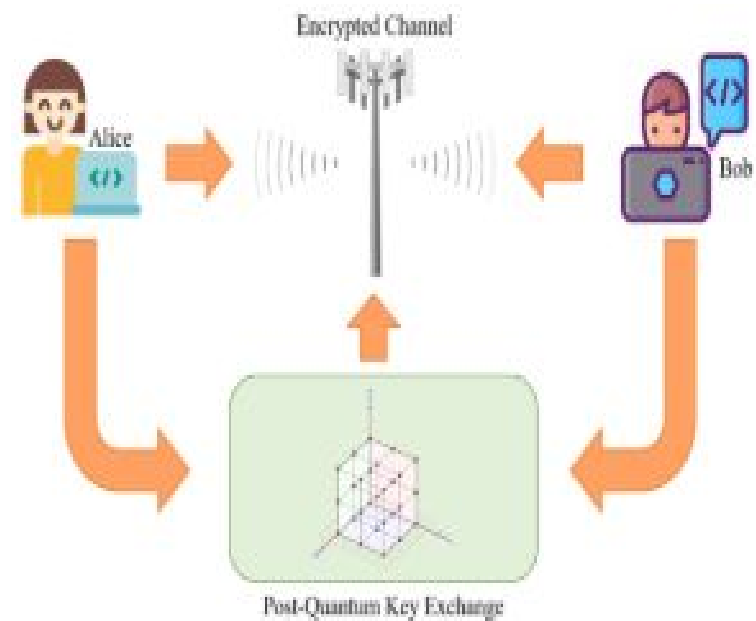
Key Candidates:

CRYSTALS-Kyber (**Key Exchange**) and

CRYSTALS-Dilithium (**Digital Signatures**)

Module-LWE for **Kyber** and

Module-SIS/LWE for **Dilithium**



Post Quantum Cryptography (PQC) secures our data by mathematics.

NIST-Selected PQC Algorithms

CRYSTALS-Kyber (Module-LWE)

This is the star for **key establishment**, the process of securely setting up a connection. It's relatively fast and has small keys. Think of it as the new quantum safe TLS handshake.

CRYSTALS-Dilithium (Module-SIS/LWE)

This is the primary candidate for **digital signatures** used for verifying identity and ensuring a message hasn't been tampered with. It's the new quantum safe signature.

FALCON (Module-SIS)

Another digital signature scheme, for when you need even smaller signatures than Dilithium.

SPHINCS+ (Hash Collisions)

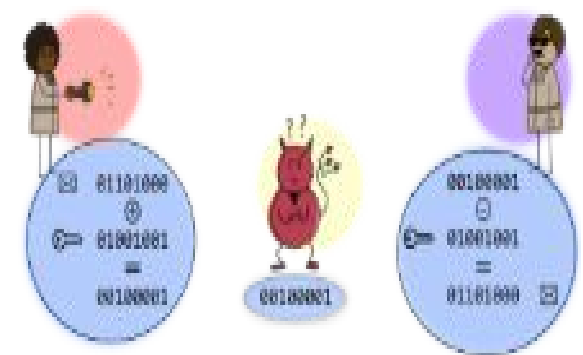
A fundamentally different, 'hash-based' signature scheme. It's considered a very conservative, safe backup option.

Quantum Cryptography

It uses principles of quantum mechanics (e.g., superposition and entanglement) to secure communication.

Quantum Key Distribution (**QKD**) allows two parties to share a secret encryption key with guaranteed security.

Unlike classical encryption (based on computational difficulty), QKD is protected by physical laws of nature.



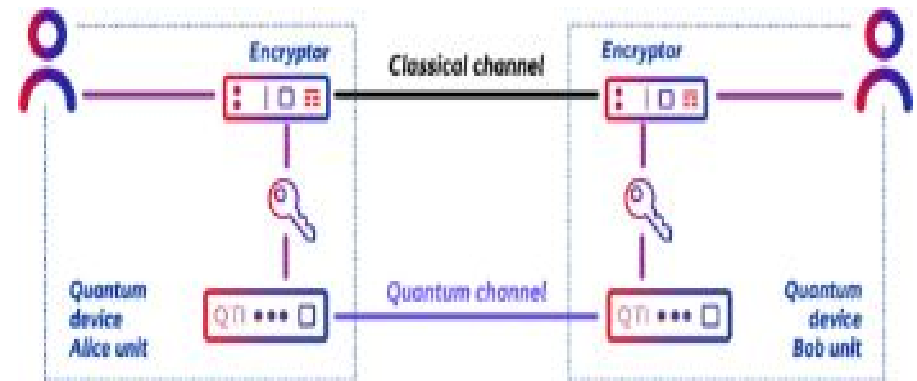
Security not by mathematics but by physics

Quantum Cryptography

The Micius satellite successfully demonstrated quantum key exchange (**QKD**) between Beijing and Vienna (7,600 km apart) in 2017

BB84 Protocol (Polarization based QKD) - Bennett & Brassard, 1984

E91 Protocol (Quantum entanglement based QKD) - Ekert, 1991



Physics based encryption for the first time in history.

Verified that entanglement based QKD works through space, paving the way for global quantum networks

Building the foundation for unbreakable global communication.

Discussion

